

Arduino Lab Experiments (1-15)

Experiment 1: RGB LED Pins (Common Cathode)

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>

/*RGB LED pins (Common Cathode)*/
#define Red_Pin 9
#define Green_Pin 10
#define Blue_Pin 11

/*Delay (in milliseconds)*/
#define COLOR_CHANGE_DELAY 1000
void setup() {
  /*Initialize the RGB LED pins as outputs*/
  pinMode(Red_Pin, OUTPUT);
  pinMode(Green_Pin, OUTPUT);
  pinMode(Blue_Pin, OUTPUT);
}

void loop() {
  /*Turn on Red, turn off Green and Blue*/
  digitalWrite(Red_Pin, HIGH);
  digitalWrite(Green_Pin, LOW);
  digitalWrite(Blue_Pin, LOW);
  delay(COLOR_CHANGE_DELAY);
```

```

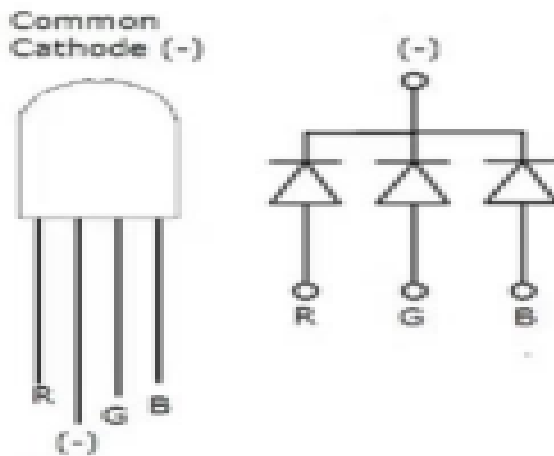
/*Turn off Red, turn on Green, turn off Blue*/
digitalWrite(Red_Pin, LOW);
digitalWrite(Green_Pin, HIGH);
digitalWrite(Blue_Pin, LOW);
delay(COLOR_CHANGE_DELAY);

/*Turn off Red, turn off Green, turn on Blue*/
digitalWrite(Red_Pin, LOW);
digitalWrite(Green_Pin, LOW);
digitalWrite(Blue_Pin, HIGH);
delay(COLOR_CHANGE_DELAY);

/*Turn ON Red, Green, Blue*/
digitalWrite(Red_Pin, HIGH);
digitalWrite(Green_Pin, HIGH);
digitalWrite(Blue_Pin, HIGH);
delay(COLOR_CHANGE_DELAY);
}

```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 2: RGB LED Pins (Common Anode)

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>

/*RGB LED pins (Common Anode)*/
#define Red_Pin 9
#define Green_Pin 10
#define Blue_Pin 11

/*Delay (in milliseconds)*/
#define COLOR_CHANGE_DELAY 1000

void setup() {
    // Initialize the RGB LED pins as
    outputs pinMode(Red_Pin, OUTPUT);
    pinMode(Green_Pin, OUTPUT);
    pinMode(Blue_Pin, OUTPUT);
}

void loop() {
    /* Pattern 1: Red */
    digitalWrite(Red_Pin, LOW); // Turn on Red (Common
    Anode) digitalWrite(Green_Pin, HIGH); // Turn off Green
    digitalWrite(Blue_Pin, HIGH); // Turn off Blue
    delay(COLOR_CHANGE_DELAY);

    /* Pattern 2: Green */
    digitalWrite(Red_Pin, HIGH); // Turn off Red
    digitalWrite(Green_Pin, LOW); // Turn on Green (Common
    Anode) digitalWrite(Blue_Pin, HIGH); // Turn off Blue
    delay(COLOR_CHANGE_DELAY);
}
```

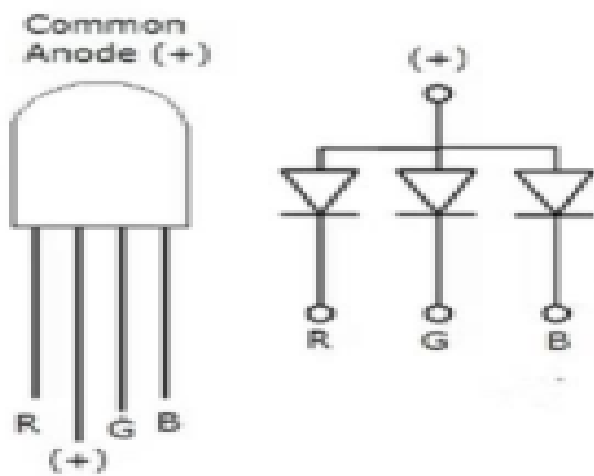
```

/* Pattern 3: Blue */
digitalWrite(Red_Pin, HIGH); // Turn off Red
digitalWrite(Green_Pin, HIGH); // Turn off Green
digitalWrite(Blue_Pin, LOW); // Turn on Blue (Common
Anode) delay(COLOR_CHANGE_DELAY);
}

/*Pattern 4: white (Red + Blue+ Green)*/
digitalWrite(Red_Pin, LOW); // Turn on Red (Common Anode)
digitalWrite(Green_Pin, LOW); // Turn on Green (Common
Anode) digitalWrite(Blue_Pin, LOW); // Turn on Blue
(Common Anode) delay(COLOR_CHANGE_DELAY);

```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 3: Buzzer

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>
const int buzzerPin = 8;
const int beepDuration = 200;
const int numBeeps = 5;

void beep(void) ;

void setup() {
    pinMode(buzzerPin, OUTPUT);
}

void loop() {
    for (int i = 0; i < numBeeps; i++) {
        beep();
        delay(500);
    }

    delay(2000);
}

void beep(void) {
    digitalWrite(buzzerPin, HIGH);
    delay(beepDuration);
    digitalWrite(buzzerPin, LOW);
    delay(100);
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 4: Push Button (Pull-up)

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>

#define buttonPin 2
#define ledPin 13

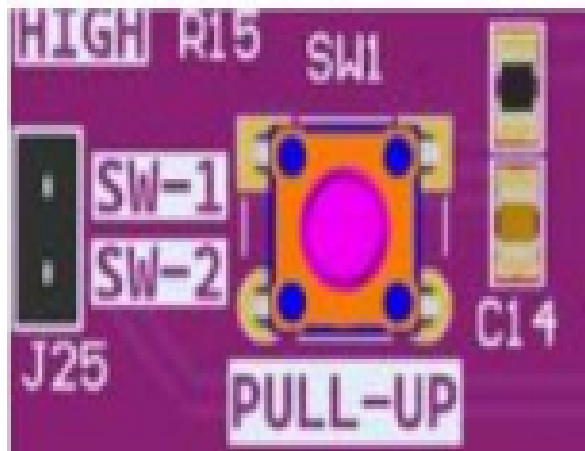
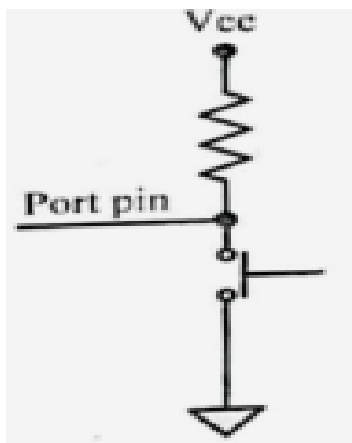
void setup() {
    pinMode(buttonPin, INPUT_PULLUP); // Enable internal pull-up
    pinMode(ledPin, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int buttonState = digitalRead(buttonPin);

    if (buttonState == LOW) { // Button pressed
        digitalWrite(ledPin, HIGH);
        Serial.println("Button Pressed");
    } else {
        digitalWrite(ledPin, LOW);
        Serial.println("Button Released");
    }

    delay(1000);
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 5: Relay

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>

// Define the relay control pin
const int relayPin = 13;

void setup() {
    // Initialize the relay control pin as an output
    pinMode(relayPin, OUTPUT);

    // Initialize serial communication
    Serial.begin(9600);
    Serial.println("Relay Control for 230V Appliance:");

    // Turn off the relay initially for safety
    digitalWrite(relayPin, LOW);
}

void loop() {
    // Prompt the user to control the appliance
    Serial.println("Enter '1' to turn ON, '0' to turn OFF:");

    while (Serial.available() <= 0);
    char command = Serial.read();
```

```
if (command == '1') {  
    Serial.println("Turning ON the Relay...");  
    digitalWrite(relayPin, HIGH); // Turn ON the relay  
} else if (command == '0') {  
    Serial.println("Turning OFF the Relay...");  
    digitalWrite(relayPin, LOW); // Turn OFF the relay  
} else {  
    Serial.println("Enter '1' to turn ON, '0' to turn OFF.");  
}  
  
// Wait for a moment  
delay(2000);  
}
```

Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 6: Sliding Switch

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>
// Define the slide switch pin
const int switchPin = 2;

void setup() {
  // Initialize the slide switch pin as an input
  pinMode(switchPin, INPUT_PULLUP);

  // Initialize serial communication
  Serial.begin(9600);
  Serial.println("Slide Switch Status:");
}

void loop() {

  int switchState = digitalRead(switchPin);
  // Check if the slide switch is in the ON position (switched) if
  (switchState == LOW) {
    Serial.println("Switch ON");
  } else {
    Serial.println("Switch OFF");
  }

  delay(500); // Add a small delay to avoid rapid toggling of status }
```

Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 7: Potentiometer

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>
const int POTENTIOMETER_PIN = A1;

void setup() {
    Serial.begin(9600);
}

void loop() {
    uint16_t sensorValue = analogRead(POTENTIOMETER_PIN);

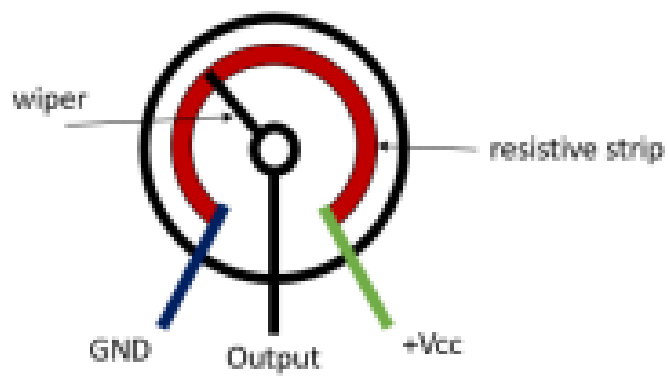
    Serial.print("RAWDATA:");
    Serial.print(sensorValue);

    float voltage = (sensorValue / 1023.0) * 5.0;

    Serial.print("Voltage: ");
    Serial.print(voltage, 2); // Print with 2 decimal places
    Serial.println(" V");

    delay(500);
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 8: LED Brightness using Potentiometer

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <Arduino.h>
const int POTENTIOMETER_PIN = A1;
const int LED_PIN = 9; // PWM pin
void setup() {
  Serial.begin(9600);
  pinMode (LED_PIN, OUTPUT);
}
void loop() {
  uint16_t sensorvalue = analogRead (POTENTIOMETER_PIN);
  Serial.print("RAWDATA:");
  Serial.print (sensorvalue);
  float voltage = (sensorvalue / 1023.0) * 5.0;
  Serial.print(" Voltage:");
  Serial.print (voltage, 2);
  Serial.println("V");
  int brightness = sensorvalue / 4;
  analogWrite (LED_PIN, brightness);
  delay(1000);
}
```

Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 9: IR Sensor

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

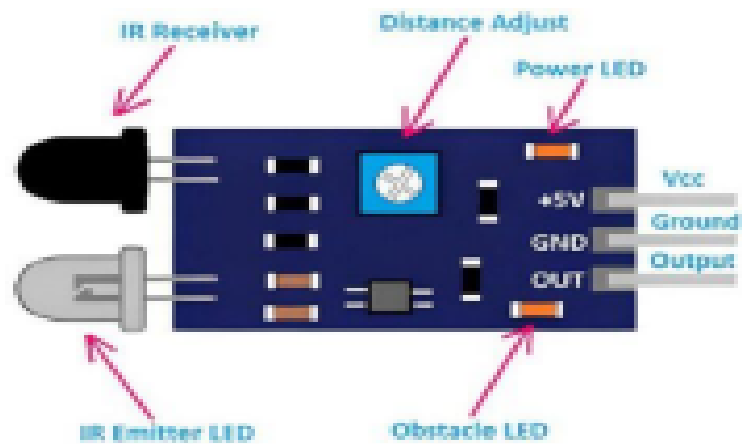
```
#include <Arduino.h>

#define IR_SENSOR_PIN 11

void setup() {
    Serial.begin(9600);
    pinMode(IR_SENSOR_PIN, INPUT);
}

void loop() {
    bool sensorValue = digitalRead(IR_SENSOR_PIN);
    Serial.print("IR Sensor Output: ");
    Serial.println(sensorValue);
    delay(1000); // 1000 milliseconds
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 10: Ultrasonic Sensor

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

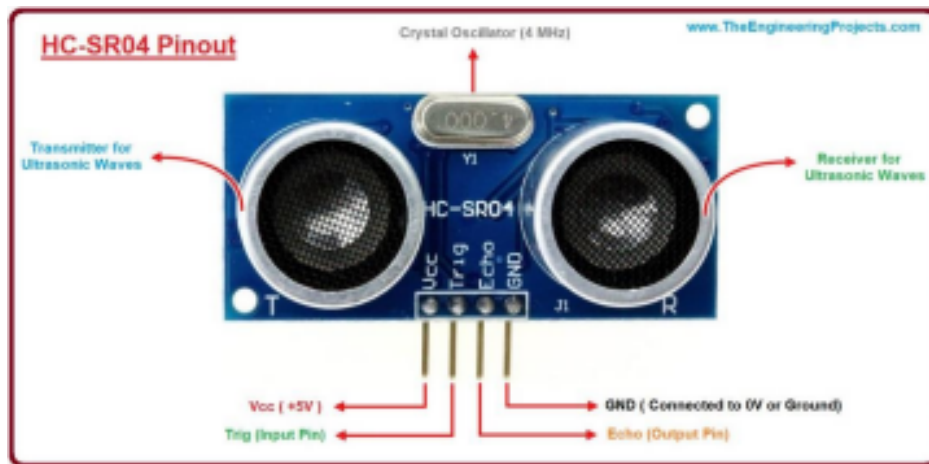
```
#include <Arduino.h>
```

```
const int TRIGGER_PIN = 8;  
const int ECHO_PIN = 9;  
const float SPEED_OF_SOUND = 343.0;
```

```
void setup() {  
    Serial.begin(9600);  
    pinMode(TRIGGER_PIN, OUTPUT);  
    pinMode(ECHO_PIN, INPUT);  
}
```

```
void loop() {  
    digitalWrite(TRIGGER_PIN, LOW);  
    delayMicroseconds(2);  
    digitalWrite(TRIGGER_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIGGER_PIN, LOW);  
    long duration = pulseIn(ECHO_PIN, HIGH);  
    float distance = (duration * 0.5 * SPEED_OF_SOUND) / 10000.0;  
    Serial.print("Distance: ");  
    Serial.print(distance, 2); // Print with 2 decimal places  
    Serial.println(" cm");  
    delay(1000);  
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 11: Temperature Sensor (LM35)

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#include <DHT11.h>
int sensorPin = A0;
float temperature;

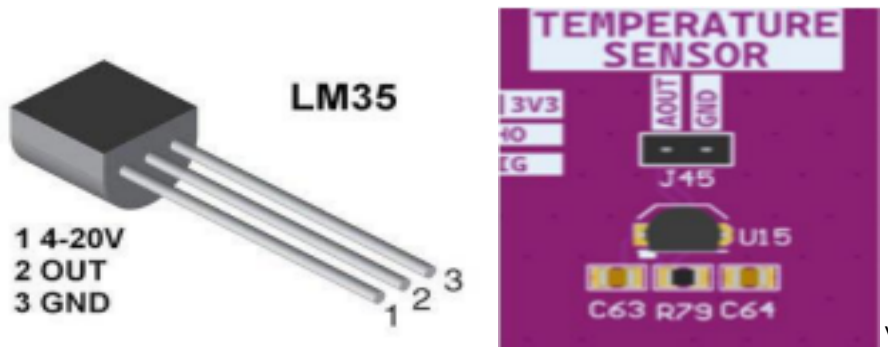
void setup() {
  Serial.begin(9600);
}

void loop() {
  int reading = analogRead(sensorPin);
  float voltage = reading * (5.0 / 1023.0);
  temperature = voltage * 100; // LM35 gives 10mV per
  °C

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");

  delay(1000);
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 12: LM35 with Voltage Reading

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

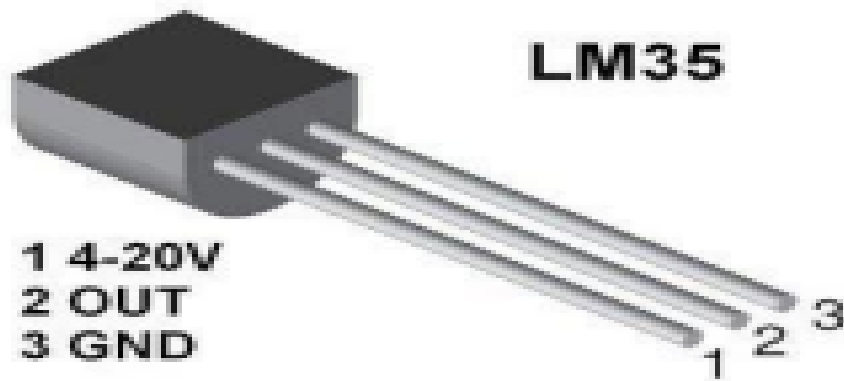
Program:

```
#include <Arduino.h> // The Arduino core
library const int TEMPERATURE_SENSOR_PIN =
A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int sensorValue =
analogRead(TEMPERATURE_SENSOR_PIN); float voltage =
(sensorValue / 1023.0) * 5.0; float
temperatureCelsius = (voltage) * 100.0;
Serial.print("Voltage: ");
Serial.print(voltage, 5);
Serial.println(" V");
Serial.print("Temperature (Celsius): ");
Serial.print(temperatureCelsius, 3);
Serial.println(" °C");
delay(1000);
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 13: Touch Sensor-Relay

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
const int SENSOR_PIN = 7;
const int RELAY = 2;
void setup() {

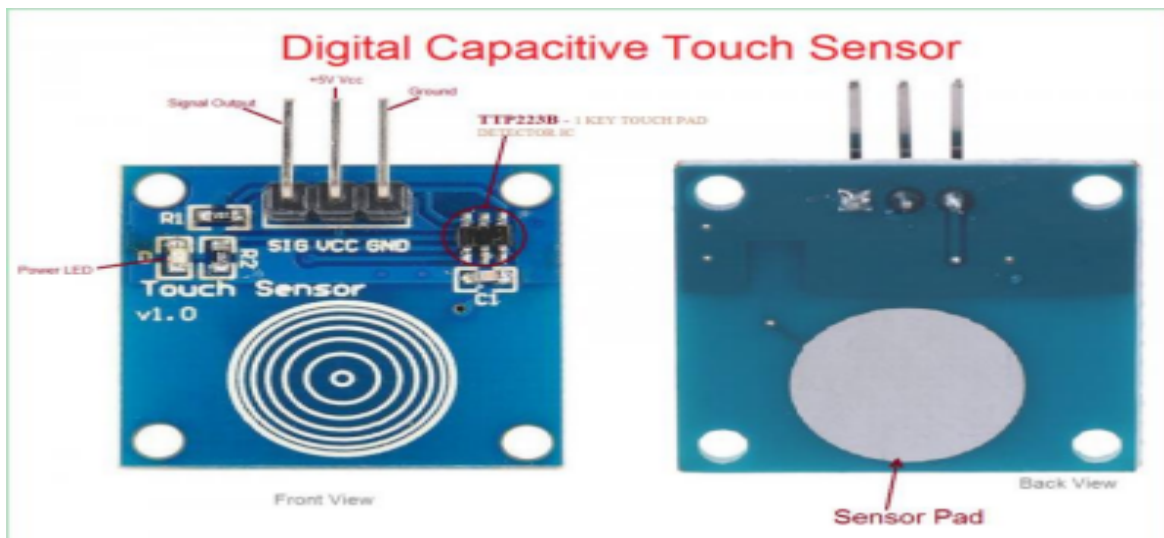
    Serial.begin(9600);
    pinMode(SENSOR_PIN, INPUT);
    pinMode(RELAY, OUTPUT);
}

void loop() {

    int state = digitalRead(SENSOR_PIN);

    digitalWrite(RELAY, state);
    if(state==1){
        Serial.println("Human is detected. open the Door!!!.");
    }
}
```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 14: LDR Street Light Control

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

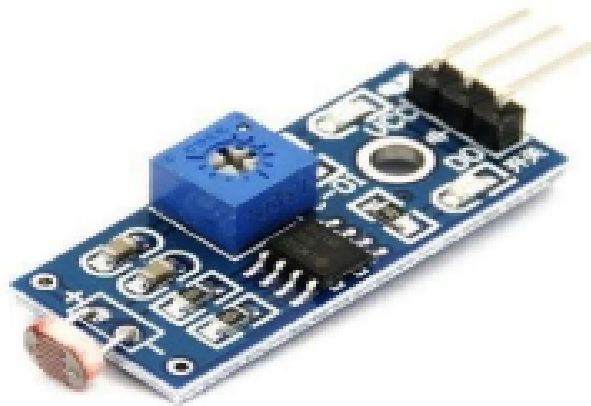
Program:

```
#define LDR 3
#define LED 13
void setup() {
  pinMode(LDR, INPUT);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  bool Sensor_Value = digitalRead(LDR);
  if(Sensor_Value ==0){
    digitalWrite(LED,0);
    Serial.println("Light is detected. It is Morning. Street lights
are turned off");
  }
  else{
    digitalWrite(LED,1);
    Serial.println("Dark... Street lights are Turned
on"); }
}
```

Related diagram:



LDR



LDR Sensor Module

Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.

Experiment 15: RGB LED with PWM

Aim: To interface and test the component using Arduino UNO and observe its operation.

Hardware Required: Arduino UNO, USB Cable, Breadboard, Jumper Wires, Component under test.

Hardware Procedure:

1. Connect the component to Arduino.
2. Verify power and ground connections.
3. Connect signal pins to Arduino I/O pins.
4. Upload the program.
5. Observe the output and verify operation.

Software Procedure:

1. Open Arduino IDE.
2. Select Arduino UNO board and COM port.
3. Write/compile the sketch.
4. Upload the sketch.
5. Observe Serial Monitor/output.

Program:

```
#define RED_LED_PIN 9
#define GREEN_LED_PIN 10
#define BLUE_LED_PIN 11
void setup()
{
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);
  pinMode(BLUE_LED_PIN, OUTPUT);
}

void loop()
{
  setColor(255, 0, 0);
  setColor(0, 255, 0);
  setColor(0, 0, 255);
  setColor(255, 255, 255);
  setColor(255, 255, 0);
  setColor(255, 0, 255);
  setColor(0, 255, 255);
}

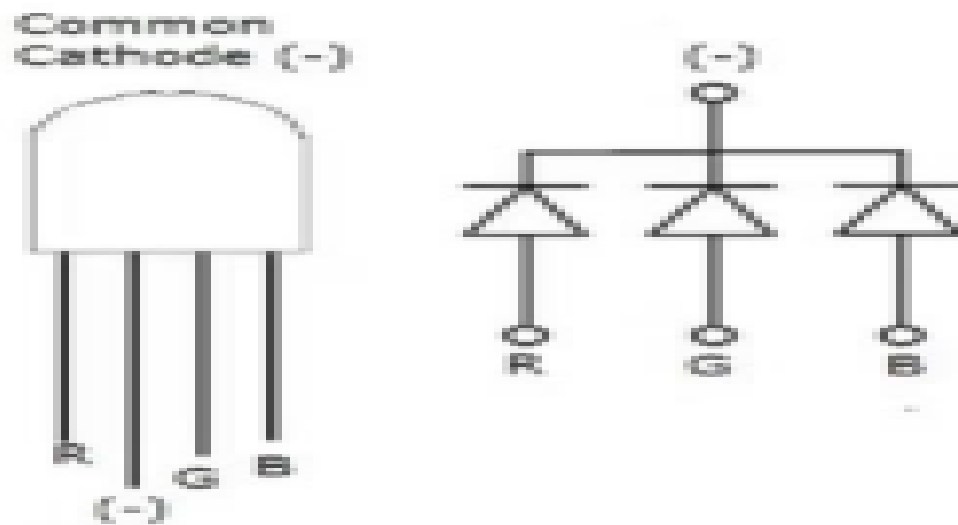
void setColor(int RED_value, int GREEN_value, int
BLUE_value) {
  analogWrite(RED_LED_PIN, RED_value);
```

```

analogWrite(GREEN_LED_PIN, GREEN_value);
analogWrite(BLUE_LED_PIN, BLUE_value);
delay(2000);
analogWrite(RED_LED_PIN, 0);
analogWrite(GREEN_LED_PIN, 0);
analogWrite(BLUE_LED_PIN, 0);
delay(1000);
}

```

Related diagram:



Result: The component was successfully interfaced with Arduino UNO and the expected output was observed.